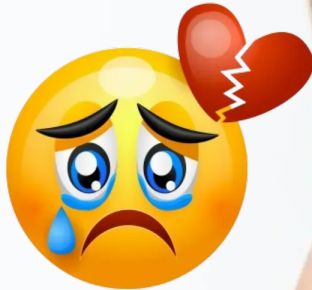
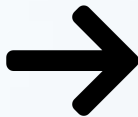


I was done with React...



Until one discovery
changed everything.

Swipe to find out...



React design patterns help developers solve common problems when building components by simplifying state management, logic, and element composition. Common patterns like custom hooks, higher-order components (HOCs), and prop-based rendering improve code consistency, reusability, and scalability. Using these patterns also makes the code easier to read, maintain, and collaborate on.

Why React Design Patterns Matter ?

Simple answer is YES!. It's because react design patterns save time, reduce costs, and make code easier to maintain by providing a clear, consistent structure. They help developers avoid issues and support long-term, sustainable development.



Repost
[@asifaliquraishy](#)

HOC Pattern

HOC (Higher Order Component) is a composition pattern for reusing any logic between components. It takes a component and returns a new component with upgraded features.

```
1 // HOC Component
2 function withLogging(WrappedComponent) {
3   return function WrappedWithLogging(props) {
4     console.log("Component is rendering");
5     return <WrappedComponent {...props} />;
6   };
7 }
8
9 // Component using HOC
10 function MyComponent() {
11   return <div>Hello, this is MyComponent!</div>;
12 }
13
14 export default withLogging(MyComponent);
```

Repost
@asifaliquraishy

Explanation

In this example, **“withLogging”** is the HOC that logs a message each time the component renders. And **“MyComponent”** is the component wrapped by the HOC to include the logging behavior. Basically, now **“MyComponent”** has added new functionality to log message.

When to use this pattern?

When need to share common functionality (like logging, authentication) between multiple components.

When you need to handle tasks like data fetching or handling UI permissions across various components.

When need to manage complex stateful logic outside components for better readability

When you need your core component logic clean by moving shared functionality outside the component.

Repost
@asifaliquuraishy

Don't Use When,

You don't need shared behavior of component. Keep it simple.

It's prefer to use hooks than HOCs. If you don't feel good with hooks, Use HOCs.

If HOCs are gonna add unnecessary complexity to your code, especially if simpler solutions exist.

Repost
@asifaliquraishy

Compound Components Pattern

Compound Component Pattern allows a main component to contain multiple child components. You can choose which child components to use and control their order. This gives you flexibility and control over the component's structure and behavior.

```
1 // Main Compound Component
2 const Dropdown = ({ children }) => <div className="dropdown">{children}</div>;
3
4 // Child Components
5 Dropdown.Toggle = ({ onClick, children }) => (
6   <button onClick={onClick} className="dropdown-toggle">
7     {children}
8   </button>
9 );
10
11 Dropdown.Menu = ({ children, isOpen }) => (
12   isOpen ? <div className="dropdown-menu">{children}</div> : null
13 );
```

Repost
@asifaliquraishy



```
1 Dropdown.Item = ({ onClick, children }) => (  
2   <div onClick={onClick} className="dropdown-item">  
3     {children}  
4   </div>  
5 );  
6  
7 // Usage Example  
8 function App() {  
9   const [isOpen, setIsOpen] = React.useState(false);  
10  
11   return (  
12     <Dropdown>  
13       <Dropdown.Toggle onClick={() => setIsOpen(!isOpen)}>  
14         Toggle Menu  
15       </Dropdown.Toggle>  
16       <Dropdown.Menu isOpen={isOpen}>  
17         <Dropdown.Item onClick={() => alert('Item 1 clicked')}>Item 1</Dropdown.Item>  
18         <Dropdown.Item onClick={() => alert('Item 2 clicked')}>Item 2</Dropdown.Item>  
19       </Dropdown.Menu>  
20     </Dropdown>  
21   );  
22 }
```

Dropdown is the main component, and **Dropdown.Toggle**, **Dropdown.Menu**, and **Dropdown.Item** are its child components. In the App component, you control the menu's visibility with **isOpen** and handle item clicks. This gives you flexible control over the dropdown's structure and behavior.

Repost
@asifaliquraishy

When to use this pattern?

When you need a component with flexible child elements that users can control and arrange.

When multiple child components share behavior or state.

When creating reusable, customizable UI elements like tabs, dropdowns, or forms.

Don't Use When,

Your component doesn't need flexible child components, keep it simple.

It adds unnecessary complexity to a straightforward component.

Your child components don't need to interact with shared logic or state.

Custom Hook Pattern

Hooks encapsulate reusable logic into functions. In React, hooks are JavaScript functions, with many built-in options available. You can also create custom hooks to share logic between components.

“useScrollPosition” is a custom hook here. It handles a logic to detect scroll position and we can use it anywhere in our app.



```
1  const ExampleComponent = () => {
2    const scrollPosition = useScrollPosition();
3
4    return (
5      <div>
6        <p>Current Scroll Position: {scrollPosition}</p>
7        {scrollPosition > 200 && <button>Back to Top</button>}
8      </div>
9    );
10 };
11
12 export default ExampleComponent;
```

Repost
@asifaliquraishy

When to use this pattern?

Use custom hooks when you need to share logic between multiple components.

To make your code more cleaner by seperating logics from UI
Manage complex stateful logic outside components for better readability

Encapsulate side effects, like data fetching or subscriptions, into a custom hook for reuse.

Don't Use When,

If the logic is only needed in one component.

It's a simple logic, that doesn't worth to become a custom hook.

There's no side effects or state. You can use regular functions.

If that makes your code harder to understand without real benefit.

Repost
@asifaliquraishy

Extensible Styles Pattern

This pattern will enable you to stylisth your components in more flexible way. Here we use dynamic CSS properties, instead directly code styles in the components. So styles can be extended and modified later we want.

```
1 // component
2 function Button({ children, className = "", style = {}, ...props }) {
3   const baseStyles = {
4     padding: "10px 20px",
5     backgroundColor: "blue",
6     color: "white",
7   };
8
9   return (
10    <button
11      className={`btn ${className}`}
12      style={{ ...baseStyles, ...style }}
13      {...props}
14    >
15      {children}
16    </button>
17  );
18 }
19
20 // Usage
21 <Button className="custom-btn" style={{ backgroundColor: "green" }}>
22   Click Me
23 </Button>;
```

Repost
@asifaliquraishy

Explanation

Button component has some base styles, like padding and background color, but you can customize them by passing `className` or `style` props. It merges the base styles with any new styles you provide. This lets you easily tweak the button's appearance without changing the component itself.

When to use this pattern?

When building components that need to be styled differently across various parts of the app.

Great for reusable buttons, cards, or forms where styles may need adjustments without duplicating code.

Ideal for creating components in a design system where consumers of the system need to customize the look.

Repost
@asifaliquraishy

Don't Use When,

Styles are fixed and don't need customization.

it adds unnecessary complexity to simple components, so stick with straightforward styling.

Repost
[@asifaliquraishy](#)

Props Getters Pattern

Props Getters Pattern lets a component provide a function to get the props needed for an element. It manages some logic while allowing the user to customize or extend the props. This makes the component flexible and reusable.

```
1  const Input = ({ getInputProps }) => {
2    return <input {...getInputProps()} />;
3  };
4
5  // Usage
6  function App() {
7    const getInputProps = () => ({
8      placeholder: "Type here",
9      onFocus: () => console.log("Focused!"),
10   });
11
12   return <Input getInputProps={getInputProps} />;
13 }
14
```

Repost
@asifaliquraishy

When to use this pattern?

When you want to share logic across components while allowing flexibility in how props are applied.

When users need to modify or extend props without losing the built-in functionality.

Ideal for creating components that are customizable but still maintain default behavior.

Don't Use When,

The component doesn't need flexible or customized props.

A component has fixed behavior and no need for extra prop flexibility, keep it simple.

Render Props Pattern

Render Props Pattern lets a component use a function (passed as a prop) to decide what to render. This allows you to share logic between components without duplicating code. The component doesn't hardcode the output but delegates it to the passed function.



```
1  const DataFetcher = ({ render }) => {  
2    const data = "Hello from Render Props!";  
3    return <div>{render(data)}</div>;  
4  };  
5  
6  // Usage  
7  function App() {  
8    return (  
9      <DataFetcher render={({data}) => <h1>{data}</h1>} />  
10   );  
11  }
```

Repost
@asifaliquraishy

Explanation

In this example, **DataFetcher** is the component with a render prop. It passes the data to the function provided by the App component, which controls how that data is displayed (in this case, inside an `<h1>` tag).

When to use this pattern?

When you want to reuse logic between components while keeping control over the rendering.

When different components need different ways to render the same data.

Don't Use When,

If it adds unnecessary complexity to simple components.

It's preferred hooks for managing state and logic if they make the code simpler.

Repost
@asifaliquraishy

State Initializer Pattern

State Initializer Pattern in React is when you initialize a component's state using props or an external function during the initial render. This allows more flexible state setup, especially if the state depends on props or requires complex logic for initialization.



```
1  const Counter = ({ initialCount = 0 }) => {  
2    const [count, setCount] = React.useState(() => initialCount);  
3    return <button onClick={() => setCount(count + 1)}>Count: {count}</button>;  
4  };  
5
```

Repost
@asifaliquraishy

When to use this pattern?

When state initialization needs a function or depends on props.

When you want to avoid recalculating state on every render, using a lazy initializer function.

Don't Use When,

State can be initialized directly without complex logic, so no need for this pattern.

It adds complexity without performance or flexibility benefits.

Control Props Pattern

Control Props Pattern in React is when a component's state is controlled by its parent through props. The parent component passes down values and callbacks to control the behavior of the child component, giving the parent full control over the child's state and actions.



```
1  const Toggle = ({ isOn, onToggle }) => {  
2    return <button onClick={onToggle}>{isOn ? "ON" : "OFF"}</button>;  
3  };  
4  
5  // Usage  
6  function App() {  
7    const [isOn, setIsOn] = React.useState(false);  
8    return <Toggle isOn={isOn} onToggle={() => setIsOn(!isOn)} />;  
9  }  
10
```

Repost
@asifaliquaishy

Explanation

Toggle component receives **isOn** and **onToggle** as control props from the parent. The parent (**App**) manages the state and passes it down, allowing full control over when the toggle is **ON** or **OFF**. This keeps the logic in the parent, while Toggle just displays and triggers the change.

When to use this pattern?

When the parent needs complete control over the child component's state.

When state needs to be shared across multiple components.

Don't Use When,

The component doesn't need external control.

The component can manage its own state, control props may not be needed.

Repost
@asifaliquraishy

State Reducer Pattern

State Reducer Pattern in React is when a component's state and its update logic are separated, often using a reducer function. This allows us for more control over how the state is managed and makes the component more flexible by providing external control over state updates.

```
1  const Toggle = ({ stateReducer }) => {
2    const [isOn, setIsOn] = React.useReducer(stateReducer, false);
3
4    return <button onClick={() => setIsOn({ type: 'toggle' })}>{isOn ? "ON" : "OFF"}</button>;
5  };
6
7  // Usage with custom state reducer
8  function App() {
9    const stateReducer = (state, action) => {
10     if (action.type === 'toggle') {
11       return !state;
12     }
13     return state;
14   };
15
16   return <Toggle stateReducer={stateReducer} />;
17 }
18
```

Repost
@asifaliquaishy

When to use this pattern?

When you need to allow external control over how state updates occur.

Ideal for managing components with complex state transitions.

Don't Use When,

The component's state management is straightforward and doesn't require custom logic.

It adds unnecessary complexity to otherwise simple components.



Hi, I'm Rosie...
I came just to teach you today.
But if you don't FOLLOW ASIF ALI,
I'll fade away forever....and
you'll never see me again